

Array Practice Problems — With Solutions

Personal Notes — Solved Practice Set

This is a curated practice booklet for array problems — the most common building blocks of DSA. Each problem includes a clean working solution, a short explanation, and where useful, a dry run or alternate approach.

Don't just read — type each solution by hand, then try a variation. Replace the input, change one condition, see what breaks. That hands-on tinkering is where the logic gets locked into your fingers.

Approach used in every solution: Declare the array → store length in a variable → use a for loop to traverse → use a tracking variable (sum, count, max, etc.) → print the result. Once this skeleton becomes automatic, every array problem feels easier.

Problem 1: Find the Average of Array Elements

Given an array of integers, calculate the average value.

Solution

```
class Main {  
    public static void main(String[] args) {  
        int[] arr = {1, 2, 3, 4, 5, 6, 7};  
        int lengthOfArr = arr.length;  
        int sum = 0;  
  
        for (int i = 0; i < lengthOfArr; i++) {  
            sum = sum + arr[i];  
        }  
  
        int avg = sum / lengthOfArr;  
        System.out.println("Average of arr is: " + avg);  
    }  
}
```

Explanation

Loop through the array, add each element to sum, then divide by the array's length.

arr = {1, 2, 3, 4, 5, 6, 7} length = 7

sum = 0 + 1 + 2 + 3 + 4 + 5 + 6 + 7 = 28

avg = 28 / 7 = 4

Output: Average of arr is: 4

Watch out: Integer division drops decimals. 28/7 happens to be exactly 4, but if sum were 29, you'd get 4 (not 4.14). Use `double avg = (double) sum / lengthOfArr;` if you want precision.

Problem 2: Multiply Each Element by 10

Modify each element of an array by multiplying it by 10, then print the result.

Solution

```
class Main {
    public static void main(String[] args) {
        int[] arr = {1, 2, 3, 4, 5, 6, 7};
        int lengthOfArr = arr.length;

        // Step 1: modify in place
        for (int i = 0; i < lengthOfArr; i++) {
            arr[i] = arr[i] * 10;
        }

        // Step 2: print the result
        for (int i = 0; i < lengthOfArr; i++) {
            System.out.print(arr[i] + " ");
        }
    }
}
```

Explanation

Two separate loops. The first one MODIFIES the array — overwriting `arr[i]` with `arr[i] * 10`. The second one PRINTS each element.

Before: {1, 2, 3, 4, 5, 6, 7}

After: {10, 20, 30, 40, 50, 60, 70}

Could be one loop: Since you're modifying AND printing, you could combine into one loop: `arr[i] = arr[i] * 10; System.out.print(arr[i] + " ");` — same result, fewer lines.

Problem 3: Search for an Element

Given an array and a target value, check if the value exists in the array.

Solution

```
class Main {
    public static void main(String[] args) {
        int[] arr = {1, 2, 3, 4, 5, 6, 7};
        int lengthOfArr = arr.length;
```

```

    int searchElement = 70;
    boolean elementFound = false;

    for (int i = 0; i < lengthOfArr; i++) {
        if (arr[i] == searchElement) {
            elementFound = true;
        }
    }

    if (elementFound) {
        System.out.println(searchElement + " is found in the array");
    } else {
        System.out.println(searchElement + " is not found in the
array");
    }
}
}

```

Explanation

This is called LINEAR SEARCH. We use a boolean flag to remember if we ever spotted the target. After the loop, we report based on the flag.

Small optimization: Once you find the element, there's no point continuing the loop. Add `break;` right after `elementFound = true;` to exit immediately. This saves time on large arrays.

Optimized Version

```

    for (int i = 0; i < lengthOfArr; i++) {
        if (arr[i] == searchElement) {
            elementFound = true;
            break;      // exit early once found
        }
    }
}

```

Problem 4: Find the Maximum Element

Given an array of integers, find the largest one.

Solution

```

class Main {
    public static void main(String[] args) {
        int[] arr = {1, 2, 30, 4, 5, 6, 7};
        int lengthOfArr = arr.length;
        int max = arr[0];

        for (int i = 0; i < lengthOfArr; i++) {
            if (arr[i] > max) {

```

```

        max = arr[i];
    }
}

System.out.println("The maximum element is: " + max);
}
}

```

Explanation

Start by assuming the first element is the max. Then loop through the rest — every time you find something bigger, update max. By the end, max holds the largest value.

```
arr = {1, 2, 30, 4, 5, 6, 7}
```

```

Start:    max = 1
i=0:      1 > 1?   no    max = 1
i=1:      2 > 1?   YES   max = 2
i=2:     30 > 2?   YES   max = 30
i=3:      4 > 30?  no    max = 30
i=4:      5 > 30?  no    max = 30
i=5:      6 > 30?  no    max = 30
i=6:      7 > 30?  no    max = 30

```

```
Result:   max = 30
```

Why start with arr[0], not 0? If all elements are negative (like {-5, -2, -8}), starting max = 0 would wrongly return 0. Always initialize max to arr[0] for safety, then start the loop at i = 1 to skip the redundant comparison.

Problem 5: Sum of Positive and Negative Elements

Given an array of integers (mix of positive and negative), find the sum of positive numbers and the sum of negative numbers separately.

Solution

```

class Main {
    public static void main(String[] args) {
        int[] arr = {1, -2, 30, 4, -5, 6, -7};
        int lengthOfArr = arr.length;
        int posSum = 0;
        int negSum = 0;

        for (int i = 0; i < lengthOfArr; i++) {
            if (arr[i] > 0) {
                posSum = posSum + arr[i];
            } else {
                negSum = negSum + arr[i];
            }
        }
    }
}

```

```

        }
    }

    System.out.println("Sum of positive elements: " + posSum);
    System.out.println("Sum of negative elements: " + negSum);
}

```

Explanation

Two tracking variables — one for positives, one for negatives. The if-else routes each element to the right bucket.

```
arr = {1, -2, 30, 4, -5, 6, -7}
```

Positive sum: $1 + 30 + 4 + 6 = 41$

Negative sum: $-2 + (-5) + (-7) = -14$

Edge case to think about: What happens with zero? In this code, zero goes into negSum (because the condition is > 0 , not ≥ 0). If you want zeros excluded, use a third branch: if positive, else if negative, else skip.

Problem 6: Count Zeros and Ones

Given an array containing only 0s and 1s, count how many of each.

Solution

```

class Main {
    public static void main(String[] args) {
        int[] arr = {1, 0, 1, 0, 1, 1, 0, 1, 0, 1};
        int lengthOfArr = arr.length;
        int countZero = 0;
        int countOne = 0;

        for (int i = 0; i < lengthOfArr; i++) {
            if (arr[i] > 0) {
                countOne = countOne + 1;
            } else {
                countZero = countZero + 1;
            }
        }

        System.out.println("Count of ones: " + countOne);
        System.out.println("Count of zeros: " + countZero);
    }
}

```

Explanation

Same pattern as Problem 5 — two counters, an if-else to decide which to increment. Because the array contains only 0 and 1, `arr[i] > 0` is the same as `arr[i] == 1`.

```
arr = {1, 0, 1, 0, 1, 1, 0, 1, 0, 1}

Ones:  1, 1, 1, 1, 1, 1  → count = 6
Zeros: 0, 0, 0, 0        → count = 4
```

Problem 7: Find the First Unsorted Element

Walk through the array left-to-right and find the first element that breaks ascending order.

Solution

```
class Main {
    public static void main(String[] args) {
        int[] arr = {1, 2, 4, 3, 5, 6, 7};
        int lengthOfArr = arr.length;

        for (int i = 0; i < lengthOfArr - 1; i++) {
            if (arr[i + 1] < arr[i]) {
                System.out.println(arr[i + 1] + " is the first unsorted
element");
                break;
            }
        }
    }
}
```

Explanation

Compare every pair of adjacent elements. The moment we find one where the right element is SMALLER than the left, we have an out-of-order pair. Print and break.

```
arr = {1, 2, 4, 3, 5, 6, 7}
      ^  ^  ^  ^
      |  |  |  |
i=0:   1  2      ✓ (2 > 1)
i=1:    2  4      ✓ (4 > 2)
i=2:    4  3      x 3 < 4 → unsorted!
```

Output: 3 is the first unsorted element

Notice the loop bound: The loop goes to `lengthOfArr - 1`, not `lengthOfArr`. That's because we look at `arr[i + 1]`, and `arr[lengthOfArr]` would be out of bounds. Always pair this kind of pattern with a -1 bound.

Shared Patterns You Can See

Looking at all 7 problems, the same skeleton shows up:

```
Step 1: Declare the array
Step 2: Get the length
Step 3: Declare tracking variables (sum, count, max, flag, etc.)
Step 4: Loop from 0 to length - 1
Step 5: Update tracking variables based on a condition
Step 6: Print the result
```

This is the universal array template. Once you internalize it, you don't write loops — you fill in this template.

More Practice — Try These Yourself

Here are 15 more array problems that follow the same patterns. Try each one BEFORE looking at hints. They get progressively harder.

Easy

- Sum of all elements (simpler version of average)
- Find the smallest (minimum) element — same as max but flipped
- Count even numbers in the array
- Count odd numbers in the array
- Find the sum of even elements only

Medium

- Find the SECOND largest element (track largest and second-largest in one pass)
- Count how many times a specific value appears in the array
- Reverse the array in place (using two pointers swapping ends)
- Check if the array is sorted in ascending order (boolean flag)
- Find the index of the maximum element
- Multiply all elements together (be careful of overflow with int)

Harder

- Move all zeros to the end while preserving the order of non-zeros
- Two-Sum: find two indices whose values add to a target
- Find the missing number from an array containing 1 to N (one missing)
- Rotate the array left by 1 position (first becomes last)

Approach for any new array problem: (1) What needs tracking? (sum, count, max, flag) → declare those before the loop. (2) Do I need just one pass or multiple? (3) What's the edge case? (empty array, single element, all negatives, all same value). If you ask these 3 questions before coding, you'll get most problems right on the first try.

Key Takeaways

- Array indices start at 0 and go to length - 1
- Always use arr.length (not a method) — it's a property
- For comparison-tracking problems (max, min), initialize with arr[0], not 0
- Use boolean flags for "did we find it?" style problems
- Add break; when you've found what you're looking for — saves time
- When indexing arr[i+1], stop the loop at length - 1 to avoid out-of-bounds
- Beware of integer division — use double when you need decimal precision
- Always think about edge cases: empty array, single element, negatives, all zeros

Common Bugs to Watch For

Bug 1: ArrayIndexOutOfBoundsException

```
for (int i = 0; i <= arr.length; i++) {    // BUG — should be <, not <=
    System.out.println(arr[i]);
}
```

Using <= goes one step too far. Use < to stop at length - 1.

Bug 2: Forgetting to initialize tracking variables

```
int sum;
for (int i = 0; i < arr.length; i++) {
    sum = sum + arr[i];    // BUG — sum was never initialized
}
```

Always initialize tracking variables before the loop — sum = 0, count = 0, max = arr[0].

Bug 3: Using == for objects (Strings in arrays)

```
String[] arr = {"hi", "bye"};
if (arr[0] == "hi") { ... }    // BUG — should use .equals()
if (arr[0].equals("hi")) { ... } // correct
```

Bug 4: Modifying array while iterating

If you change array elements inside a loop AND those changes affect the loop condition, you can get weird off-by-one or skipped elements. Best practice: either modify or read, not both at the same time.

Keep at it! These patterns become muscle memory once you've solved 20-30 array problems. Then larger DSA topics like sliding window, two-pointer, and binary search will feel like natural extensions.